

MVLU COLLEGE
A.I. Practical No.8

Aim

- To implement the K-Nearest Neighbors (K-NN) algorithm for classification.
- Train a K-NN model using a given dataset and evaluate its performance for different values of K.
- Calculate the accuracy of the model on test data and analyze the results.

Code:-

```
Practical 8.py - D:\Haripr\AI\Practical 8.py (3.11.5)
File Edit Format Run Options Window Help
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
print("Hariprasad Vishwakarma T127")
df = pd.read_csv(r"F:\Downloads\Most Runs - 2016.csv")
print("Dataset loaded successfully!")
print("\nFirst 5 rows:")
print(df.head())
print("\nDataset shape:")
print(df.shape)
print("\nColumn names:")
print(df.columns.tolist())

if "Player" in df.columns:
    df = df.drop("Player", axis=1)
if "POS" in df.columns:
    df = df.drop("POS", axis=1)

if "HS" in df.columns:
    df["HS"] = df["HS"].astype(str).str.replace(" ", "", regex=False)
    df["HS"] = pd.to_numeric(df["HS"], errors="coerce")

for column in df.columns:
    df[column] = pd.to_numeric(df[column], errors="coerce")

df = df.dropna()

def performance_category(runs):
    if runs <= 250:
        return "Low"
    elif runs < 450:
        return "Medium"
    else:
        return "High"

df["Performance"] = df["Runs"].apply(performance_category)

X = df.drop(["Runs", "Performance"], axis=1)
y = df["Performance"]
```

MVLU COLLEGE

A.I. Practical No.8

```
Practical 8.py - D:\Har\AI\Practical 8.py (3.13.5)
File Edit Format Run Options Window Help
X = df.drop(["Runs", "Performance"], axis=1)
y = df["Performance"]

encoder = LabelEncoder()
y = encoder.fit_transform(y)

print("\nNumber of features:", X.shape[1])
print("Number of samples:", X.shape[0])
print("Target classes:", encoder.classes_)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train_scaled, y_train)

print("\nK-NN model trained successfully!")
print("K value: 5")

y_pred = model.predict(X_test_scaled)

print("\nFirst 15 Actual Values:")
print(encoder.inverse_transform(y_test[:15]))

print("\nFirst 15 Predicted Values:")
print(encoder.inverse_transform(y_pred[:15]))

accuracy = accuracy_score(y_test, y_pred)
error = 1 - accuracy
```

```
Practical 8.py - D:\Har\AI\Practical 8.py (3.13.5)
File Edit Format Run Options Window Help
accuracy = accuracy_score(y_test, y_pred)
error = 1 - accuracy

print("\n=====")
print("K-NN MODEL PERFORMANCE")
print("=====")
print("Accuracy:", round(accuracy * 100, 2), "%")
print("Error:", round(error * 100, 2), "%")

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=encoder.classes_,
    zero_division=0
))

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

classes = encoder.classes_

plt.figure(figsize=(8, 6))
plt.imshow(cm, interpolation="nearest")
plt.title("K-NN Confusion Matrix")
plt.xlabel("Predicted Performance")
plt.ylabel("Actual Performance")
plt.colorbar()
plt.xticks(range(len(classes)), classes)
plt.yticks(range(len(classes)), classes)

for i in range(len(classes)):
    for j in range(len(classes)):
        plt.text(j, i, cm[i, j], ha="center", va="center")

plt.tight_layout()

k_values = range(1, 16)
accuracy_values = []
```

Hariprasad Vishwakarma
T127
TYCS

MVLU COLLEGE

A.I. Practical No.8

```
Practical 8.py - D:/Hari/AI/Practical 8.py (3.13.5)
File Edit Format Run Options Window Help

for i in range(len(classes)):
    for j in range(len(classes)):
        plt.text(j, i, cm[i, j], ha="center", va="center")

plt.tight_layout()

k_values = range(1, 16)
accuracy_values = []

for k_value in k_values:
    model = KNeighborsClassifier(n_neighbors=k_value)
    model.fit(X_train_scaled, y_train)
    predictions = model.predict(X_test_scaled)
    score = accuracy_score(y_test, predictions)
    accuracy_values.append(score)

best_index = accuracy_values.index(max(accuracy_values))
best_k = list(k_values)[best_index]
best_accuracy = accuracy_values[best_index]

print("\n=====")
print("BEST K VALUE")
print("=====")
print("Best K:", best_k)
print("Best Accuracy:", round(best_accuracy * 100, 2), "%")

plt.figure(figsize=(8, 5))
plt.plot(
    list(k_values),
    accuracy_values,
    marker="o"
)
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("K-NN: K Value vs Accuracy")
plt.xticks(list(k_values))
plt.grid(True)
plt.tight_layout()

plt.show()
```

Output:-

```
"IDLE Shell 3.13.5"
File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: D:/Hari/A.I/Practical 8.py =====
Hariprasad Vishwakarma T127
Dataset loaded successfully!

First 5 rows:
   POS  Player  Mat  Inns  NO  Runs  ...  BF  SR  100  50  4s  6s
0    1  Virat Kohli  16   16   4   973  ...  640 152.03  4   7  83  38
1    2  David Warner  17   17   3   848  ...  560 151.42  0   9  88  31
2    3  AB de Villiers  16   16   3   687  ...  407 168.79  1   6  57  37
3    4  Shikhar Dhawan  17   17   4   501  ...  429 116.78  0   4  51   8
4    5  Gautam Gambhir  15   15   2   501  ...  411 121.89  0   5  54   6

[5 rows x 14 columns]

Dataset shape:
(136, 14)

Column names:
['POS', 'Player', 'Mat', 'Inns', 'NO', 'Runs', 'HS', 'Avg', 'BF', 'SR', '100', '50', '4s', '6s']

Number of features: 11
Number of samples: 136
Target classes: ['High' 'Low' 'Medium']

Training samples: 108
Testing samples: 28

K-NN model trained successfully!
K value: 5

First 15 Actual Values:
['Low' 'Low' 'Low' 'Low' 'Medium' 'Low' 'Low' 'Low' 'Low' 'Low'
 'Low' 'High' 'Low' 'Low']

First 15 Predicted Values:
['Low' 'Low' 'Low' 'Low' 'Medium' 'Low' 'Low' 'Low' 'Low' 'Low'
 'Low' 'High' 'Low' 'Low']

=====
K-NN MODEL PERFORMANCE
```

Hariprasad Vishwakarma
T127
TYCS

MVLU COLLEGE

A.I. Practical No.8

```

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

Testing samples: 28

K-NN model trained successfully!
K value: 5

First 15 Actual Values:
['Low' 'Low' 'Low' 'Low' 'Medium' 'Low' 'Low' 'Low' 'Low' 'Low' 'Low'
'Low' 'High' 'Low' 'Low']

First 15 Predicted Values:
['Low' 'Low' 'Low' 'Low' 'Medium' 'Low' 'Low' 'Low' 'Low' 'Low' 'Low'
'Low' 'High' 'Low' 'Low']

=====
K-NN MODEL PERFORMANCE
=====
Accuracy: 100.0 %
Error: 0.0 %

Classification Report:
              precision    recall  f1-score   support

      High         1.00        1.00        1.00         2
      Low          1.00        1.00        1.00        23
      Medium       1.00        1.00        1.00         3

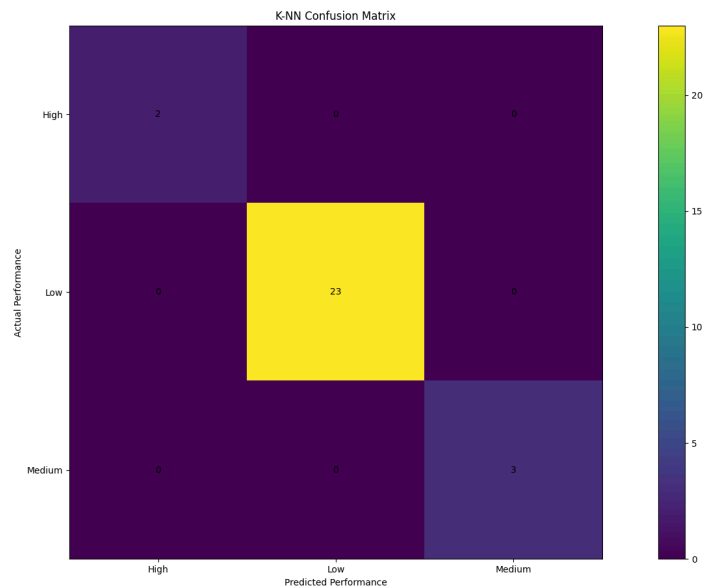
   accuracy          1.00
  macro avg          1.00
 weighted avg          1.00

Confusion Matrix:
[[ 2  0  0]
 [ 0 23  0]
 [ 0  0  3]]

=====
BEST K VALUE
=====
Best K: 1
Best Accuracy: 100.0 %

```

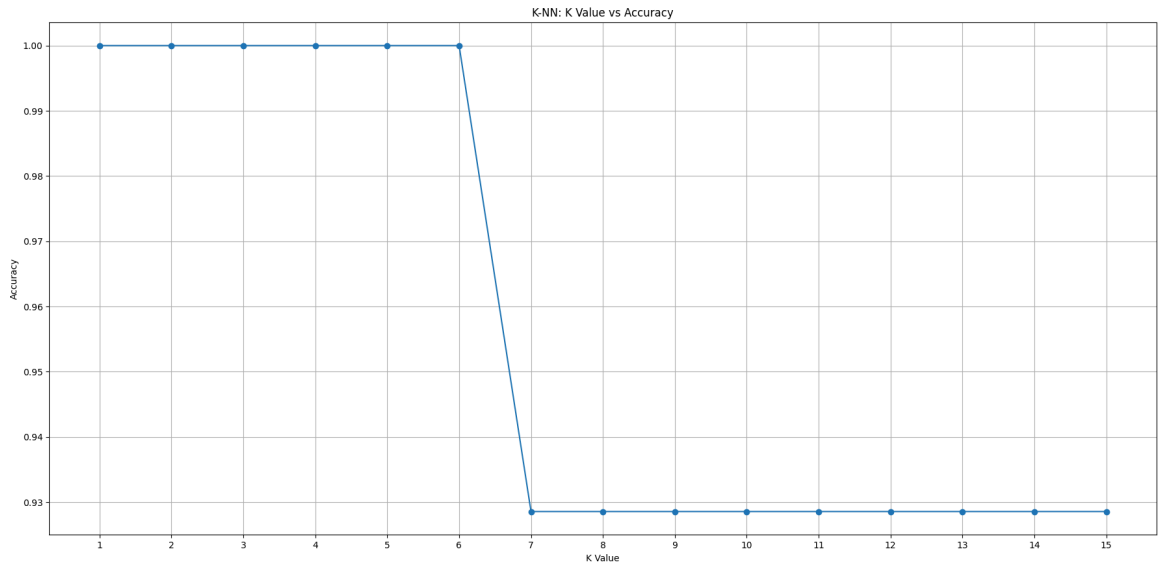
Figure 1



MVLU COLLEGE

A.I. Practical No.8

Figure 2



Observation

- 1. Confusion Matrix:** The confusion matrix shows that most IPL players are correctly classified into their respective performance categories, with fewer incorrect predictions.
- 2. Training Loss Curve:** The training loss decreases as the number of iterations increases, indicating that the neural network is learning effectively and improving its performance.